TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión y Facturación para Kiosco

Grupo: 10

Integrantes: Santino Valentín Arias, Gabriel Maximiliano Perdomo, Cristian Tapia.

Docente coordinador: Sebastián Bruselario.

Fecha: 31/10/2026.

Índice

1. Introducción y descripción del negocio 4
1.1. Introducción 4
1.2. Problemática del negocio 4
1.3. Objetivos del proyecto 4
1.4. Alcance del sistema 4
2. Modelado de datos NoSQL 5
2.1. Estructura de la base de datos 5
2.2. Esquemas de las colecciones 5
2.3. Relaciones mediante Embedding y Linking 7
2.4. Implementación de baja lógica 8
3. Infraestructura Cloud y configuración de MongoDB Atlas 8
3.1. Creación del clúster M0 8
3.2. Configuración de usuarios y acceso de red 8
3.3. Conexión mediante Node.js y Mongoose 10
4. Desarrollo de la aplicación y operaciones CRUD 12
4.1. Tecnologías y estructura de la aplicación 12
4.2. Operación CREATE: registro de productos 12
4.3. Operación READ: consulta de productos 13
4.4. Operación UPDATE: modificación de productos 13
4.5. Operación DELETE y verificación de baja lógica 14
5. Gestión y registro de ventas 15
5.1. Implementación del módulo de ventas 15
5.2. Registro y almacenamiento de ventas 15
5.3. Actualización automática del stock 16
5.4. Consulta de ventas registradas 17
6. Automatización de respaldos de la base de datos 18
6.1. Implementación del mecanismo de respaldo 18
6.2. Ejecución del respaldo 18
6.3. Verificación de los archivos generados 18
6.4. Recuperación de datos y políticas RTO/RPO 19

7. Código fuente e instrucciones de ejecución 19

7.1. Organización de los archivos 19

7.2. Configuración de la conexión 20

7.3. Instrucciones de ejecución 20

8. Conclusión y reflexión técnica 21

1. Introducción y descripción del negocio

1.1. Introducción

El presente Trabajo Práctico Integrador de Bases de Datos II tiene como objetivo aplicar los conocimientos adquiridos sobre modelado NoSQL, persistencia de información en la nube y desarrollo de aplicaciones conectadas a bases de datos.

Para ello, se desarrolló un sistema de gestión para un kiosco utilizando MongoDB Atlas, JavaScript, Node.js y Mongoose. La aplicación funciona mediante una interfaz de consola que permite administrar productos, consultar existencias y registrar ventas. Además, el proyecto incorpora scripts para generar respaldos de la información almacenada.

1.2. Problemática del negocio

La administración de un kiosco requiere mantener un control actualizado de productos, precios, stock y ventas. Cuando esta información se gestiona manualmente, pueden producirse errores en el inventario y dificultades para consultar registros anteriores.

Como solución, se propuso desarrollar una aplicación que centralice los datos del negocio en una base de datos NoSQL, facilitando su administración y conservación mediante operaciones controladas.

1.3. Objetivos del proyecto

El objetivo general consiste en diseñar e implementar un sistema de gestión conectado a MongoDB Atlas, que permita administrar información comercial mediante operaciones CRUD y mecanismos de respaldo.

Para alcanzar este propósito se establecieron los siguientes objetivos específicos:

- Diseñar tres colecciones relacionadas: categorias, productos y ventas.
- Aplicar estrategias de Embedding y Linking para organizar los documentos.
- Implementar operaciones CRUD y bajas lógicas desde una aplicación de consola.
- Registrar ventas, validar existencias y actualizar automáticamente el stock.
- Almacenar la información en MongoDB Atlas.
- Desarrollar scripts para automatizar los respaldos de la base de datos.

1.4. Alcance del sistema

El sistema permite gestionar productos, categorías y ventas mediante una aplicación ejecutada desde la terminal. Incluye operaciones de creación, consulta, modificación, baja lógica, registro de ventas y consultas de auditoría.

También incorpora validaciones básicas y scripts de respaldo compatibles con Windows y Linux/macOS.

El proyecto se concentra en demostrar la integración entre una aplicación y una base de datos NoSQL en la nube, sin incorporar una interfaz gráfica ni funcionalidades contables avanzadas.

2. Modelado de datos NoSQL

2.1. Estructura de la base de datos

Para almacenar la información del kiosco se utilizó MongoDB, una base de datos NoSQL orientada a documentos que emplea el formato BSON.

La base de datos, denominada kiosko_db, se organizó en tres colecciones relacionadas:

- categorias: almacena los rubros comerciales del negocio.
- productos: contiene el catálogo de artículos, precios, existencias y referencias a categorías.
- ventas: conserva las operaciones comerciales y los detalles de los productos vendidos.

Esta organización permite administrar los datos del negocio y mantener un registro de sus operaciones.

2.2. Esquemas de las colecciones

Colección categorias: Almacena el nombre y la descripción de cada categoría, junto con su estado lógico y las fechas de creación y actualización.

```
{
  _id: ObjectId("67168a1f8b4a2c1098e10001"),
  nombre: "Golosinas",
  descripcion: "Chocolates, alfajores y caramelos",
  activo: true,
  createdAt: ISODate("2026-10-23T14:00:00.000Z"),
  updatedAt: ISODate("2026-10-23T14:00:00.000Z")
}
```

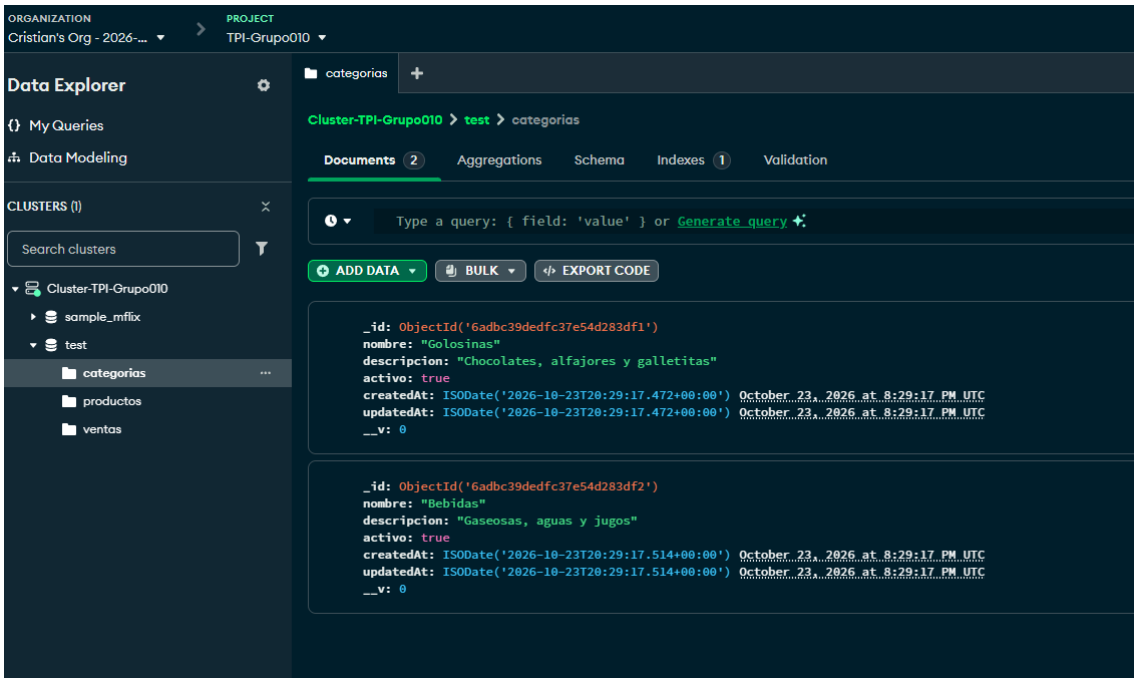


Figura 1. Visualización de la colección categorias en MongoDB Atlas.

Colección productos: Contiene el nombre del artículo, precio, stock, categoría asociada y estado lógico. La relación con categorias se establece mediante el campo categoria_id.

```
{
  _id: ObjectId("67168a1f8b4a2c1098e10002"),
  nombre: "Alfajor Triple Chocolate",
  precio: 1450,
  stock: 45,
  categoria_id: ObjectId("67168a1f8b4a2c1098e10001"),
  activo: true,
  createdAt: ISODate("2026-10-23T14:05:00.000Z"),
  updatedAt: ISODate("2026-10-23T14:30:00.000Z")
}
```

Colección ventas: Registra la fecha, el método de pago, el importe total y los productos comercializados mediante un arreglo de documentos embebidos.

```
{
  _id: ObjectId("67168a1f8b4a2c1098e10003"),
  fecha: ISODate("2026-10-23T15:10:00.000Z"),
  metodo_pago: "Efectivo",
  total: 2900,
  activo: true,
```

```
  items: [
    {
      producto_id: ObjectId("67168a1f8b4a2c1098e10002"),
      nombre: "Alfajor Triple Chocolate",
      cantidad: 2,
      precio_unitario: 1450
    }
  ],
  createdAt: ISODate("2026-10-23T15:10:00.000Z"),
  updatedAt: ISODate("2026-10-23T15:10:00.000Z")
}
```

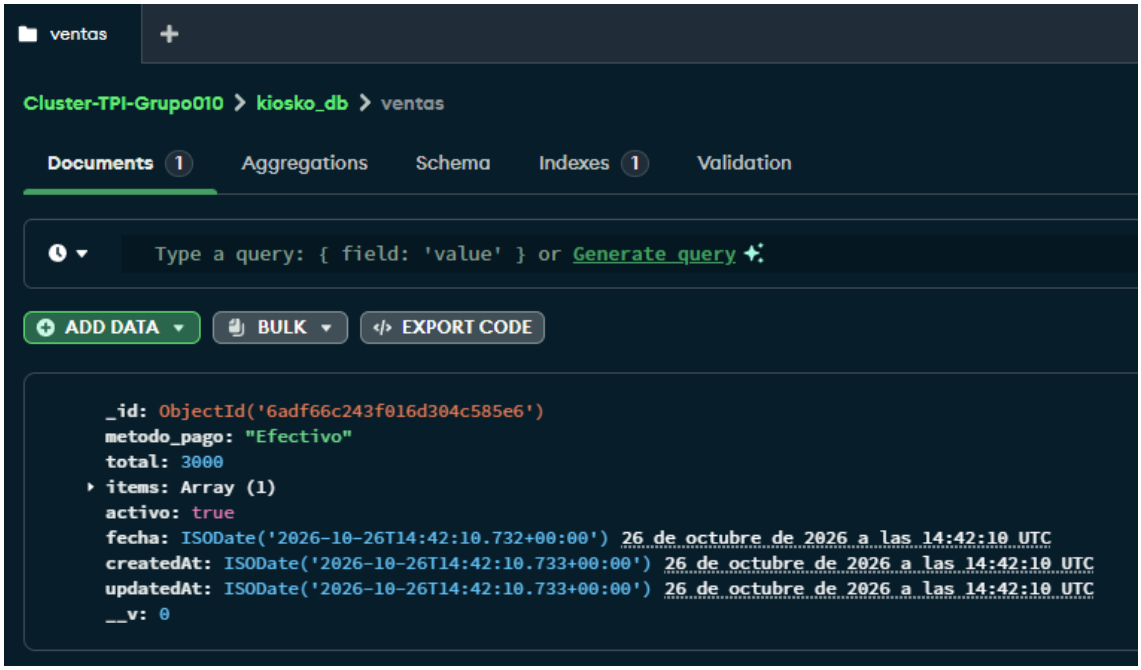


Figura 2. Documento de venta con productos embebidos en MongoDB Atlas.

2.3. Relaciones mediante Embedding y Linking

Para relacionar las colecciones se utilizaron dos estrategias de modelado NoSQL.

Linking (referenciación): se aplicó entre productos y categorias. Cada producto almacena el identificador de su categoría mediante un campo de tipo ObjectId, evitando duplicar información y facilitando las modificaciones.

Embedding (incrustación): se aplicó en la colección ventas, donde cada documento contiene los datos de los productos comercializados. Esta estrategia permite conservar los precios y cantidades correspondientes al momento de la operación, incluso si posteriormente cambia la información del catálogo.

2.4. Implementación de baja lógica

Para evitar la eliminación física de registros se incorporó el atributo booleano activo, que inicialmente posee el valor true.

Cuando un producto es dado de baja, el sistema modifica su estado a false mediante el operador $set. Las consultas habituales aplican el filtro { activo: true }, mientras que las consultas de auditoría permiten recuperar todos los documentos, incluidos los inactivos.

Este mecanismo conserva la información para futuras verificaciones.

3. Infraestructura Cloud y configuración de MongoDB Atlas

3.1. Creación del clúster M0

Para implementar la persistencia de los datos en la nube se utilizó MongoDB Atlas, una plataforma que permite administrar bases de datos MongoDB de forma remota.

Se creó un clúster gratuito M0 para alojar la base de datos kiosko_db. Esta infraestructura permitió almacenar los documentos y ejecutar operaciones desde la aplicación desarrollada en Node.js.

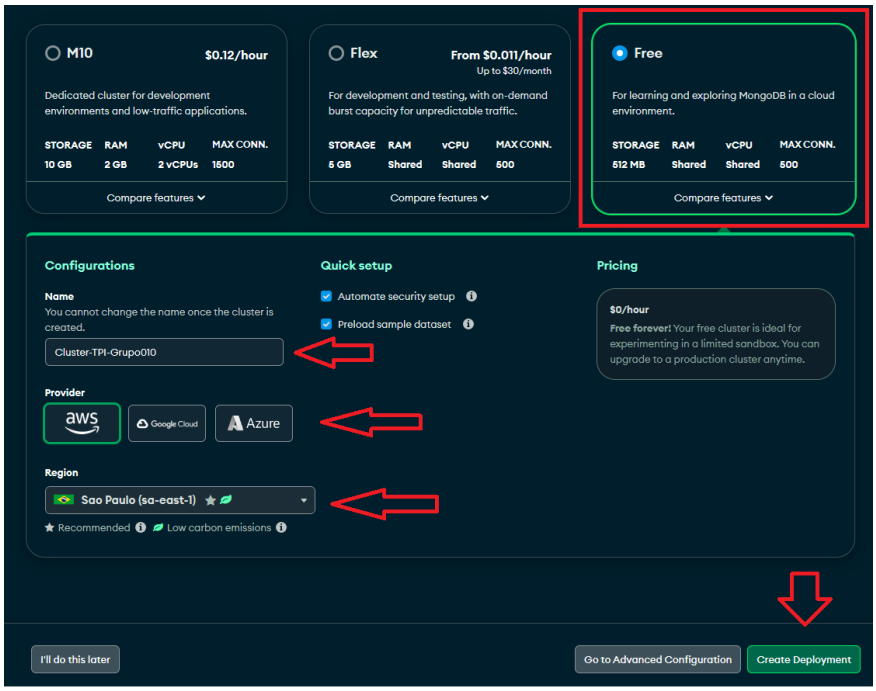


Figura 3. Configuración del clúster gratuito M0 en MongoDB Atlas.

3.2. Configuración de usuarios y acceso de red

Para permitir la conexión con la base de datos se creó un usuario con permisos de lectura y escritura desde la sección Database Access de MongoDB Atlas.

Durante la creación del usuario se configuraron las credenciales de autenticación y los permisos necesarios para que la aplicación pudiera realizar operaciones de lectura y escritura sobre la base de datos.

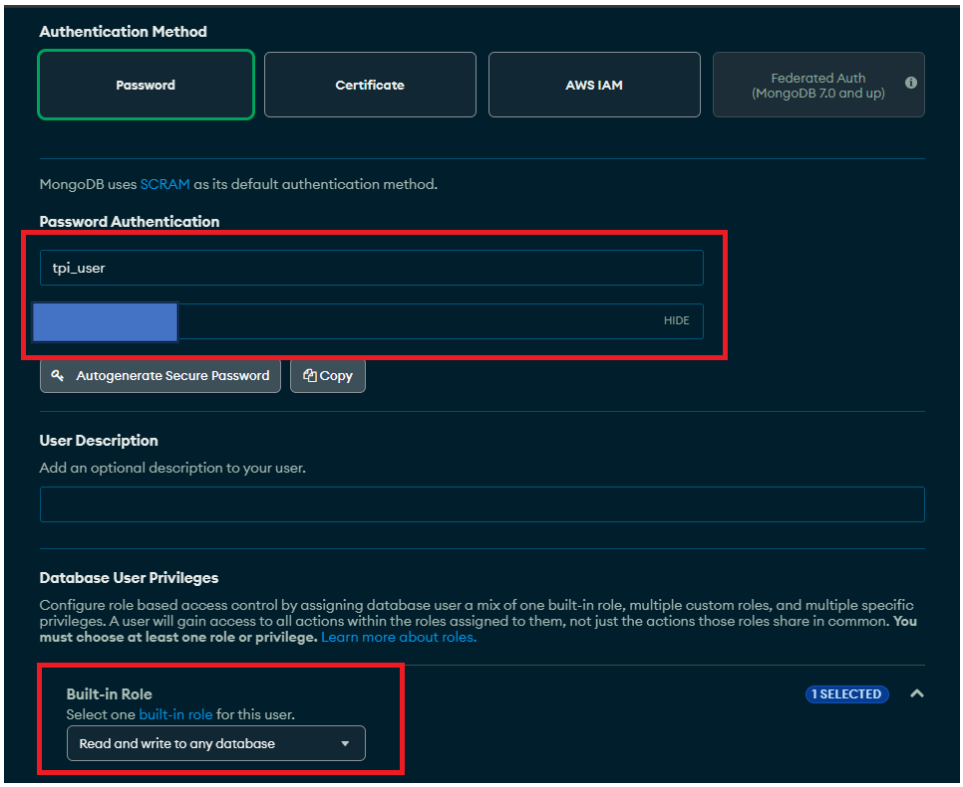


Figura 4. Configuración de los permisos del usuario de MongoDB Atlas.

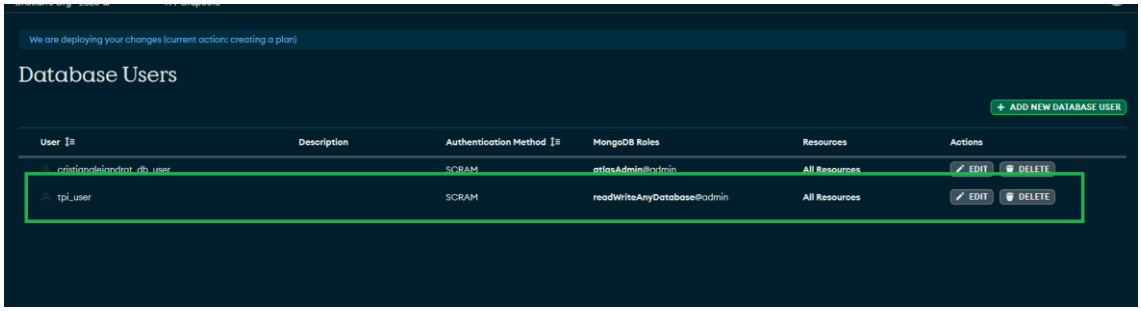


Figura 5. Configuración del usuario y sus permisos de acceso.

Posteriormente, se configuró la sección Network Access de MongoDB Atlas para habilitar las conexiones remotas. Para ello, se incorporó una regla de acceso mediante la dirección IP 0.0.0.0/0, que permite conexiones desde cualquier dirección IP. Finalmente, se verificó que la regla quedara registrada correctamente en la plataforma.

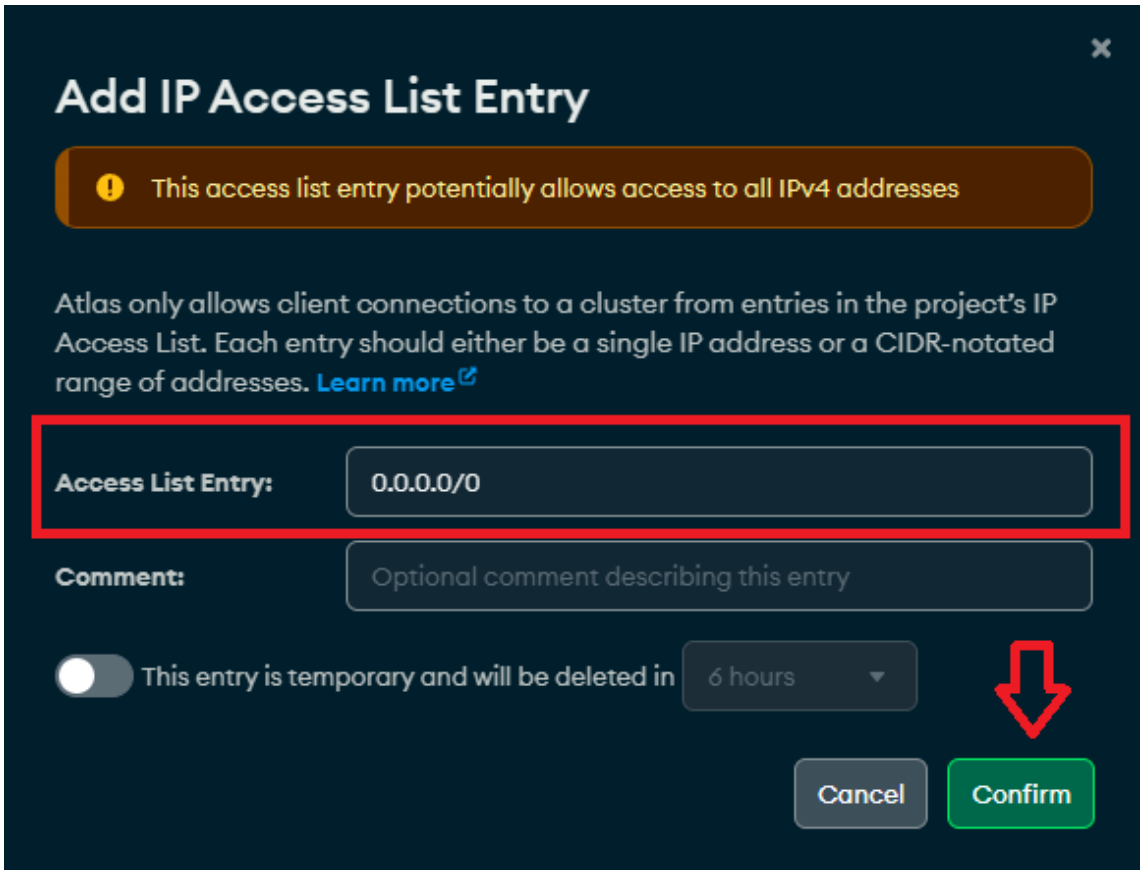


Figura 6. Incorporación de la regla de acceso IP en MongoDB Atlas.

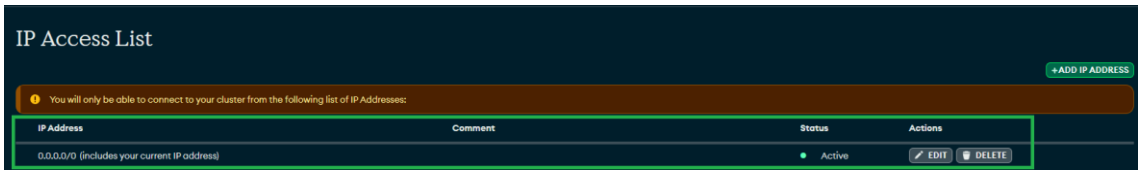


Figura 7. Configuración del acceso de red al clúster.

Esta configuración facilitó las pruebas del proyecto. Para un entorno de producción se recomienda restringir el acceso a direcciones IP autorizadas.

3.3. Conexión mediante Node.js y Mongoose

La comunicación entre la aplicación y MongoDB Atlas se implementó utilizando Mongoose, una biblioteca que permite definir modelos y ejecutar operaciones sobre documentos.

La cadena de conexión se almacena en un archivo .env mediante la variable MONGODB_URI, cargada a través de dotenv. De esta manera, las credenciales permanecen separadas del código fuente.

La conexión se establece mediante el método mongoose.connect(), utilizando la cadena almacenada en la variable de entorno MONGODB_URI. El siguiente fragmento simplificado corresponde al código de conexión implementado en el archivo app.js:

```
import mongoose from 'mongoose';

import dotenv from 'dotenv';
```

```
dotenv.config();

const uri = process.env.MONGODB_URI?.trim();

await mongoose.connect(uri);
```

Esta configuración permite establecer la conexión sin incorporar directamente las credenciales al código fuente.

Durante la integración se presentaron dificultades con la resolución DNS SRV. Para solucionarlas, se utilizó una cadena de conexión estándar que especifica directamente los servidores del conjunto de réplicas de MongoDB Atlas.

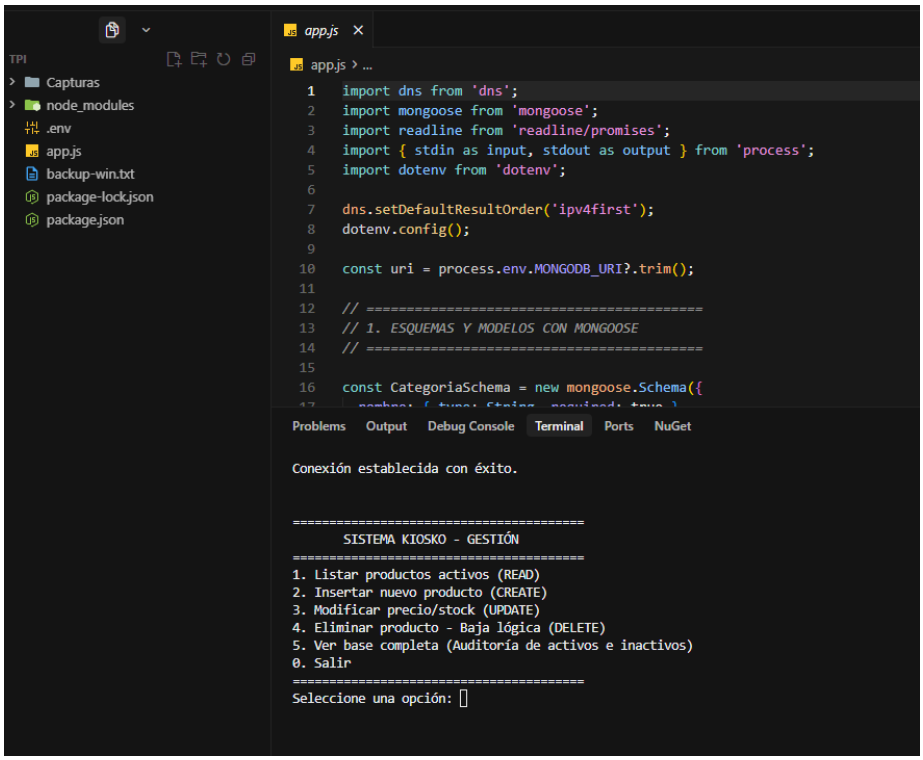


Figura 8. Verificación de la conexión entre la aplicación y MongoDB Atlas.

Esta configuración permitió establecer la comunicación entre el entorno local y la base de datos remota.

Para comprobar la persistencia de los datos, se accedió al explorador de colecciones de MongoDB Atlas y se verificó la presencia de los documentos almacenados. Esta comprobación permitió confirmar que los registros gestionados mediante la aplicación se encontraban disponibles en la base de datos Cloud.

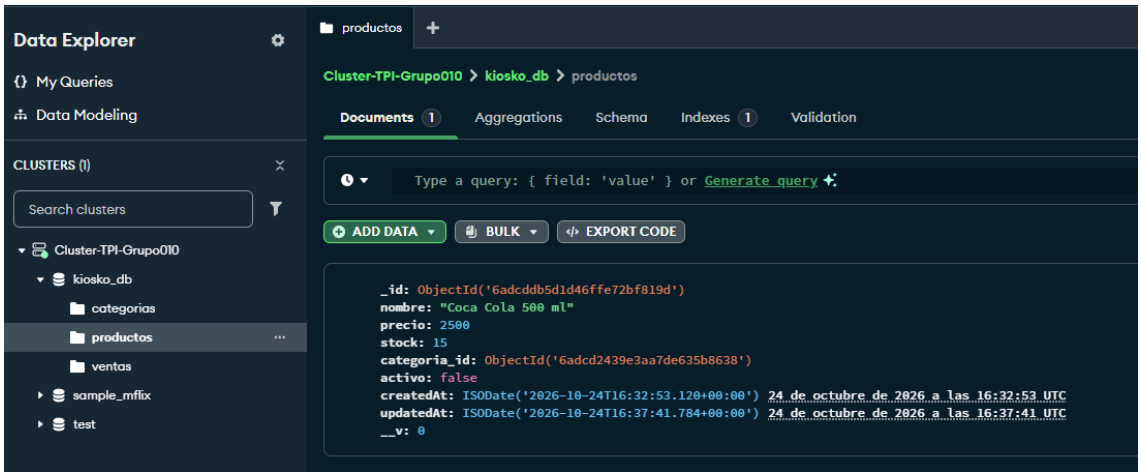


Figura 9. Verificación de los documentos persistidos en MongoDB Atlas.

4. Desarrollo de la aplicación y operaciones CRUD

4.1. Tecnologías y estructura de la aplicación

La aplicación se desarrolló en JavaScript mediante Node.js y utiliza Mongoose para interactuar con MongoDB Atlas.

El proyecto se organiza principalmente en dos archivos:

- app.js: contiene los modelos, las funciones CRUD, la conexión con MongoDB y el menú interactivo.
- ventas.mjs: incorpora las funciones para registrar y consultar ventas.

También se utilizaron dotenv para las variables de entorno y readline/promises para la interacción mediante la terminal.

El sistema ofrece un menú que permite administrar productos, consultar registros, registrar ventas y realizar operaciones de auditoría.

4.2. Operación CREATE: registro de productos

La operación CREATE permite incorporar nuevos productos al catálogo.

El sistema solicita seleccionar una categoría e ingresar el nombre, precio y cantidad inicial. Después de validar los datos, utiliza el método save() de Mongoose para almacenar el documento en MongoDB Atlas.

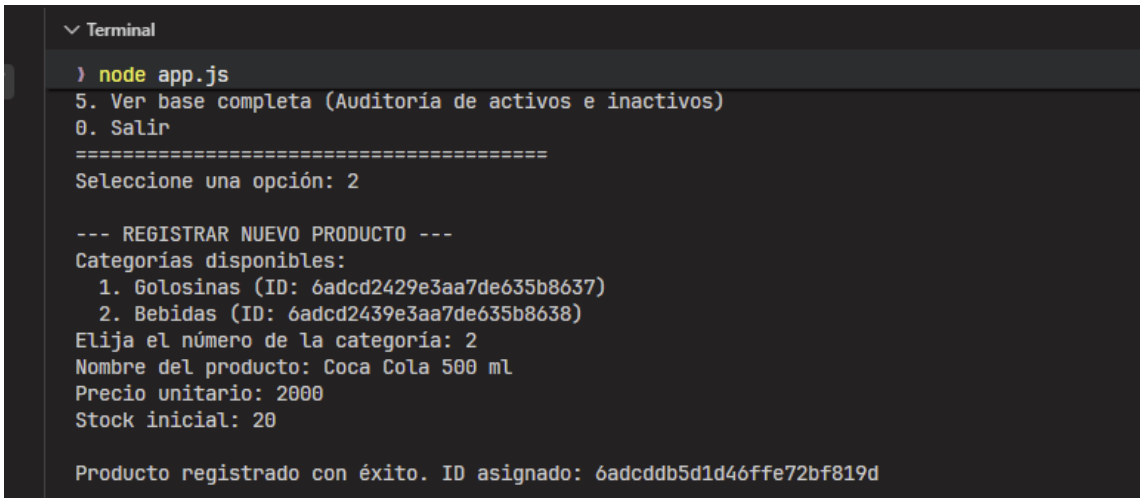


Figura 10. Registro de un producto mediante la operación CREATE.

4.3. Operación READ: consulta de productos

La operación READ recupera los productos activos mediante el método find() y el filtro { activo: true }.

Además, utiliza populate() para obtener el nombre de la categoría relacionada. Los resultados muestran el identificador, nombre, precio, stock y categoría de cada artículo.

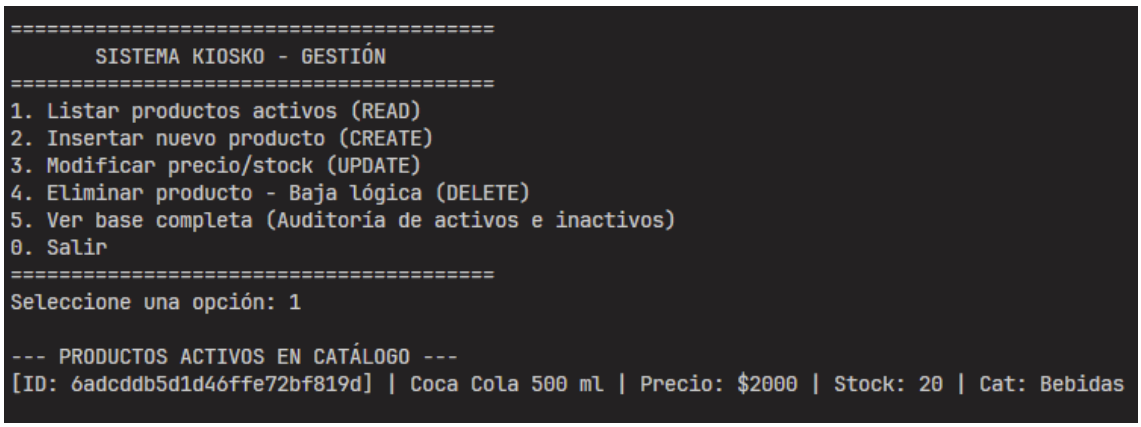


Figura 11. Consulta de productos activos mediante la operación READ.

4.4. Operación UPDATE: modificación de productos

La operación UPDATE permite modificar el precio y las existencias de un producto previamente registrado.

Para ello, se valida el identificador ObjectId y se utiliza findOneAndUpdate() junto con el operador $set, actualizando únicamente los campos indicados.



Figura 12. Actualización del precio y stock de un producto.

4.5. Operación DELETE y verificación de baja lógica

La eliminación de productos se implementó mediante baja lógica, modificando el atributo activo a false sin borrar físicamente el documento.

Esta operación utiliza findByIdAndUpdate() y permite conservar la información almacenada.

La baja lógica se implementó mediante la siguiente función:

```
async function bajaLogicaProducto(id) {
  return await Producto.findByIdAndUpdate(
    id,
    { $set: { activo: false } },
    { new: true }
  );
}
```

La función modifica únicamente el estado del documento, conservando los datos del producto en la colección.



Figura 13. Ejecución de la operación DELETE mediante baja lógica.

Para verificar su funcionamiento, se incorporó una consulta de auditoría que recupera productos activos e inactivos, comprobando que los documentos dados de baja continúan registrados.

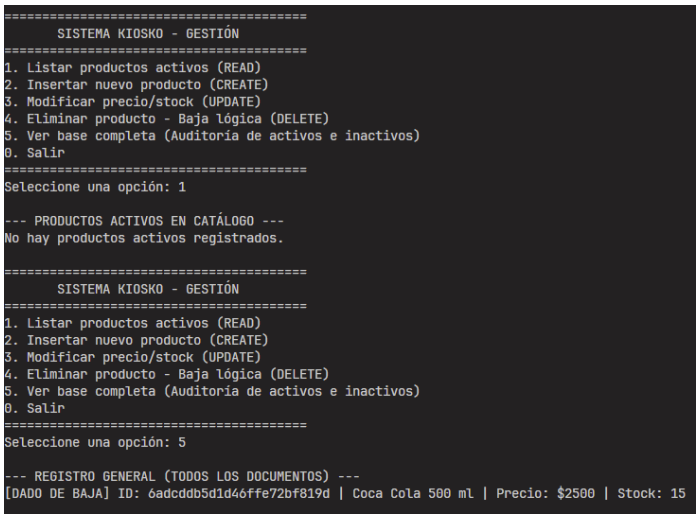


Figura 14. Verificación de la baja lógica y conservación del registro mediante auditoría.

De esta manera, se comprobaron las operaciones CRUD requeridas y la persistencia de sus resultados en MongoDB Atlas.

5. Gestión y registro de ventas

5.1. Implementación del módulo de ventas

Para complementar la gestión de productos se desarrolló el módulo ventas.mjs, encargado de registrar y consultar las operaciones comerciales del kiosco.

La aplicación permite seleccionar un producto activo con stock disponible, ingresar la cantidad solicitada y elegir el método de pago entre efectivo y transferencia.

Antes de confirmar la operación, el sistema verifica que la cantidad sea válida y no supere las existencias disponibles. Posteriormente, calcula el importe total y solicita la confirmación del usuario.

5.2. Registro y almacenamiento de ventas

Como parte de las pruebas iniciales, se realizó una consulta a la colección de ventas desde el menú de la aplicación. El sistema informó que no existían operaciones registradas, permitiendo verificar el estado inicial antes de realizar la primera venta.

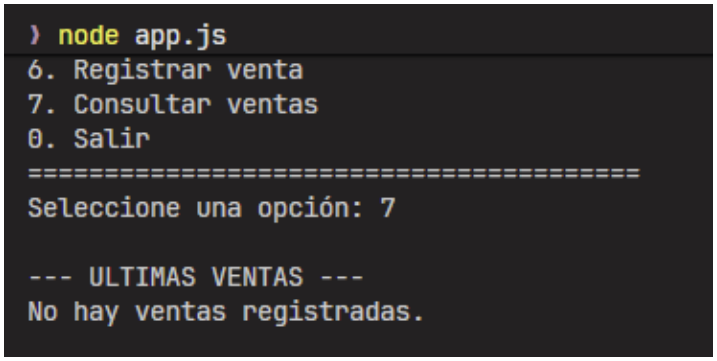


Figura 15. Consulta inicial del módulo de ventas sin operaciones registradas.

Una vez confirmada la operación, el sistema crea un documento en la colección ventas, almacenando la fecha, método de pago, importe total y detalle del producto comercializado.

El registro utiliza documentos embebidos para conservar el nombre, cantidad y precio unitario aplicado al momento de la venta.

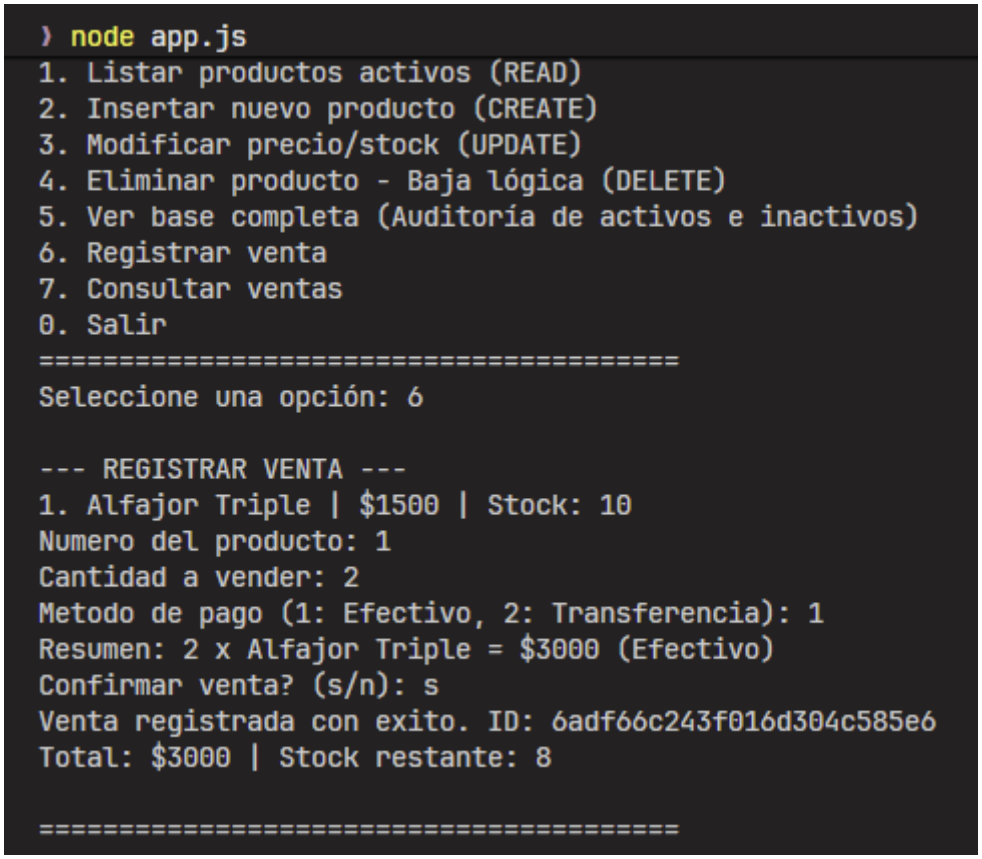


Figura 16. Registro exitoso de una venta desde la aplicación de consola.

5.3. Actualización automática del stock

Al confirmar una venta, la aplicación utiliza findOneAndUpdate() junto con el operador $inc para descontar del inventario las unidades comercializadas.

La actualización incluye una condición que verifica la disponibilidad del producto, evitando realizar descuentos superiores al stock existente.

Si se produce un error durante el almacenamiento de la venta, el sistema intenta restituir las unidades descontadas.

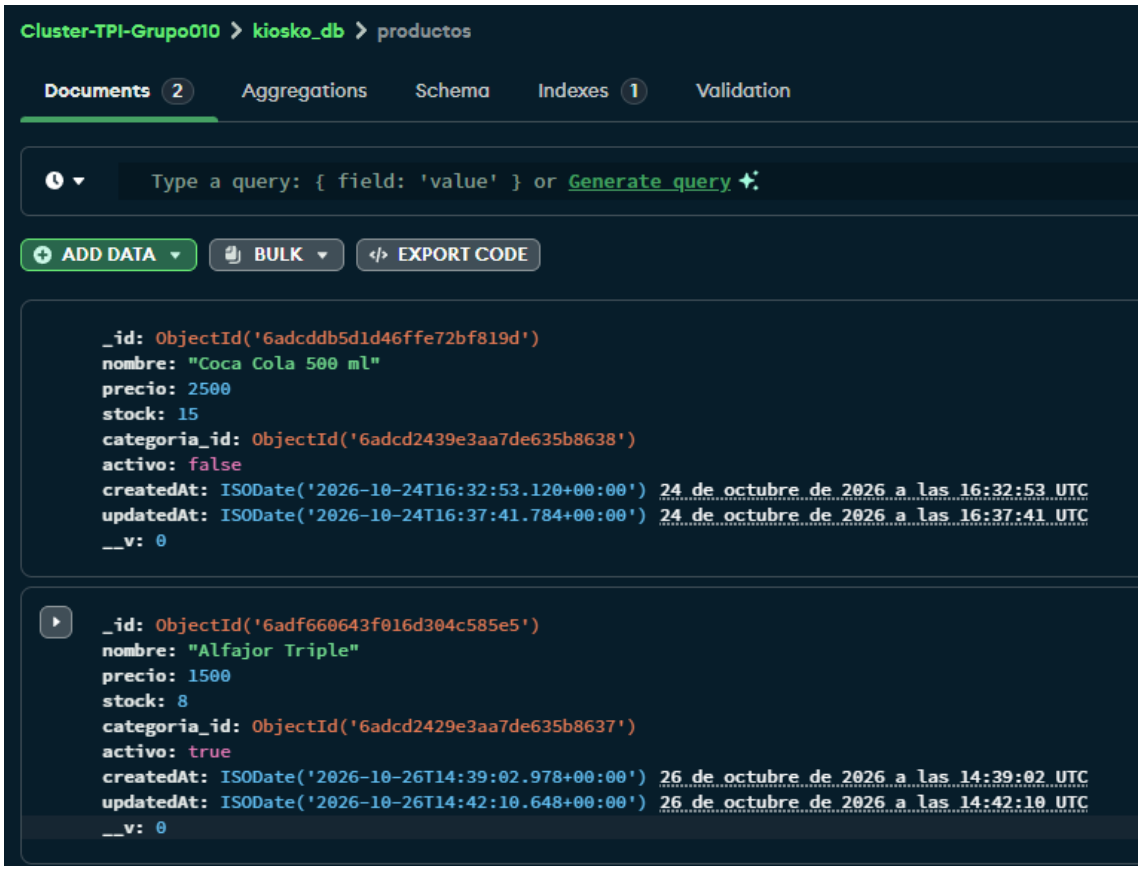


Figura 17. Verificación de la reducción del stock después de registrar una venta.

5.4. Consulta de ventas registradas

La aplicación incorpora una opción que permite consultar las últimas diez ventas activas, ordenadas desde la más reciente.

Para cada operación se muestran su identificador, fecha, método de pago, importe total y detalle de los productos vendidos.

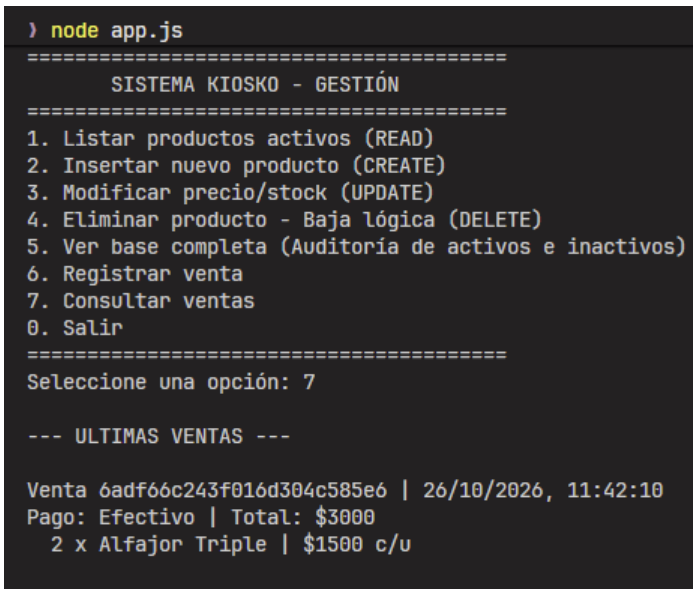


Figura 18. Consulta de ventas registradas mediante la aplicación.

Las pruebas permitieron comprobar el registro de operaciones, la actualización del inventario y la persistencia de la información en MongoDB Atlas.

6. Automatización de respaldos de la base de datos

6.1. Implementación del mecanismo de respaldo

Con el objetivo de contribuir a la conservación y recuperación de la información almacenada en MongoDB Atlas, se desarrollaron scripts que permiten generar copias de respaldo mediante la herramienta mongodump.

Se implementaron dos versiones: backup.bat para Windows y backup.sh para Linux/macOS. Ambos scripts obtienen la cadena de conexión desde el archivo .env y utilizan rutas relativas al proyecto para almacenar los respaldos dentro del directorio resguardos_tpi.

6.2. Ejecución del respaldo

Al ejecutar el script, se obtiene la fecha actual y se crea automáticamente una carpeta con el formato AAAA-MM-DD, destinada a almacenar los archivos de respaldo de esa jornada.

Posteriormente, se ejecuta mongodump, que se conecta a MongoDB Atlas y exporta la información de las colecciones. El script también verifica el resultado de la operación e informa si finalizó correctamente o si se produjo un error.



Figura 19. Ejecución del script de respaldo mediante la herramienta mongodump.

6.3. Verificación de los archivos generados

Una vez completado el procedimiento, se verificó la creación del directorio de respaldo y la existencia de los archivos exportados.

La herramienta genera archivos .bson, que contienen los documentos de las colecciones, y archivos .metadata.json, que almacenan información complementaria.

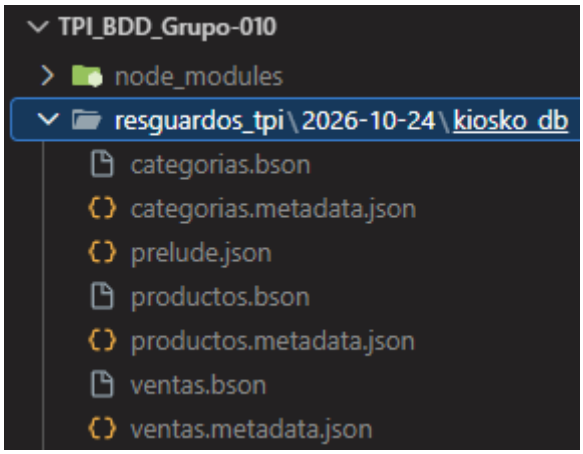


Figura 20. Verificación de los archivos BSON y de metadatos generados durante el respaldo.

La comprobación permitió verificar que la información de la base de datos se exportó correctamente. Este mecanismo facilita la conservación de copias de seguridad y proporciona una alternativa para la recuperación de datos mediante mongorestore en caso de ser necesario.

6.4. Recuperación de datos y políticas RTO/RPO

Los respaldos generados mediante mongodump permiten disponer de copias de los documentos almacenados en MongoDB Atlas. Ante una eventual pérdida de información, estos archivos pueden utilizarse para realizar una restauración mediante la herramienta mongorestore.

Para evaluar la recuperación de información se consideran dos indicadores:

RPO (Recovery Point Objective): representa el período máximo de información que podría perderse entre el último respaldo disponible y una falla. Si se establece una política de respaldo diario, el RPO objetivo sería de 24 horas.

RTO (Recovery Time Objective): representa el tiempo máximo previsto para restablecer el funcionamiento del sistema después de una interrupción.

En el proyecto se implementaron los scripts necesarios para generar respaldos y se verificó la existencia de los archivos exportados. Sin embargo, la ejecución periódica automática y el tiempo efectivo de restauración no fueron medidos, por lo que no se establece un RTO comprobado.

7. Código fuente e instrucciones de ejecución

7.1. Organización de los archivos

Los principales archivos que componen la implementación son:

- app.js: conexión con MongoDB Atlas, modelos de Mongoose, operaciones CRUD y menú interactivo.
- ventas.mjs: registro y consulta de operaciones comerciales.

- package.json: configuración del proyecto y dependencias.
- .env-example: plantilla de configuración de la conexión.
- backup.bat: script de respaldo para Windows.
- backup.sh: script de respaldo para Linux/macOS.

El código fuente completo de la aplicación se encuentra disponible en el repositorio de GitHub referenciado a este informe y forma parte de los archivos correspondientes a la entrega del Trabajo Práctico Integrador. En este informe se documentan las funcionalidades principales, las decisiones técnicas y las evidencias de su funcionamiento.

7.2. Configuración de la conexión

Para ejecutar la aplicación se requiere disponer de Node.js, las dependencias del proyecto y acceso a un clúster de MongoDB Atlas.

La cadena de conexión se configura mediante la variable MONGODB_URI, almacenada en un archivo .env local. Como referencia, el repositorio incluye .env-example, que permite identificar la estructura de configuración sin publicar credenciales reales.

La configuración de conexión se realiza mediante la variable de entorno MONGODB_URI, utilizando una cadena de conexión estándar que especifica los servidores del conjunto de réplicas de MongoDB Atlas.

Ejemplo de configuración:

```
MONGODB_URI=mongodb://<usuario>:<contraseña>@<servidor-1>:27017/kiosko_db
```

La cadena presentada tiene carácter ilustrativo y no constituye una conexión funcional directa al proyecto. En la implementación real se especifican los tres servidores del conjunto de réplicas de MongoDB Atlas, junto con los parámetros de autenticación y conexión necesarios. Las credenciales se almacenan en el archivo .env, evitando incorporarlas directamente al código fuente.

7.3. Instrucciones de ejecución

Para utilizar el sistema se deben realizar los siguientes pasos:

1. Descargar o clonar el repositorio del proyecto.
2. Ejecutar npm install para instalar las dependencias necesarias.
3. Crear el archivo .env a partir de .env-example y configurar la cadena de conexión correspondiente.
4. Ejecutar node app.js desde la terminal.
5. Seleccionar las operaciones disponibles mediante el menú interactivo.

Para generar un respaldo, se debe disponer de MongoDB Database Tools y ejecutar backup.bat en Windows o backup.sh en Linux/macOS.

Repositorio del proyecto: https://github.com/CTapia10/TPI_BDD_Grupo-010

8. Conclusión y reflexión técnica

El desarrollo del Trabajo Práctico Integrador permitió aplicar los conocimientos adquiridos sobre bases de datos NoSQL, modelado de documentos y persistencia de información en entornos Cloud.

Mediante MongoDB Atlas se implementó una base de datos compuesta por tres colecciones relacionadas, utilizando estrategias de Embedding y Linking según las necesidades del sistema. Asimismo, se desarrolló una aplicación de consola capaz de ejecutar operaciones CRUD, aplicar bajas lógicas y registrar ventas con actualización del inventario.

Desde el punto de vista técnico, el proyecto permitió comprender el funcionamiento de la comunicación cliente-servidor. La aplicación desarrollada en Node.js actúa como cliente, mientras que MongoDB Atlas proporciona el servicio remoto de almacenamiento. Mongoose facilita esta interacción mediante modelos y operaciones asíncronas, permitiendo enviar consultas y recibir resultados a través de la conexión de red.

Durante el desarrollo también se presentaron dificultades relacionadas con la conexión DNS SRV, que se resolvieron utilizando una cadena de conexión directa a los servidores de MongoDB Atlas. Esta experiencia permitió profundizar en la importancia de la configuración de red y la resolución de problemas de conectividad.

Finalmente, la implementación de scripts de respaldo permitió incorporar mecanismos de conservación de la información mediante herramientas nativas de MongoDB.

En conclusión, se lograron integrar los principales conceptos abordados en la asignatura, desarrollando una solución funcional que demuestra la relación entre el modelado NoSQL, la persistencia en la nube, las operaciones sobre documentos y la automatización de tareas de administración.